CSC226 Assignment 1

Dryden Bryson

February 21, 2026

Introduction

Seperate chaining and open adressing are both collision management techniques used when implementing hash tables. Separate chaining relies on linked lists, initialized at each entry in the hash table, thus when a collision occurs it can be added to the linked list. Open addressing allows elements to be indexed to different addresses in the table when a collision occurs. Seperate chaining is a simpler technique to implement, which one could assume that the simplicity would indicate a more efficient implementation but it would turn out that, that is not the case.

Methodology

To compare the two techniques, we will use experimental analysis. We will implement both techniques and then run them on the same data set. The following factors in the experimental analysis were considered and chose:

- **Programming Language**: C has been chosen as the language to implement the two schemes as the language has minimal abstraction overhead, and direct control of the memory layout since the two techniques can be heavily affected by the memory layout and allocation. The code was compiled using the `gcc` compiler.
- **Domain of experimentation**: Since we want to get an idea of the performance in all scenarios, all combinations of the following have been considered.
  - **Load Factor**: Is tested from 0.5 to 0.95 in increments of 0.05
  - **Map Size**: For this experiment fixed map sizes of 2^n where $n = \{5, 6, 7, ..., 24\}$

  Additionally each of the following possible combinations was executed 10 times to get an average time for each combination.
- **Data selection:** The keys are generated using C's psuedo-random number generator using a fixed seed to ensure that the experiments are consistent for both techniques.
- **Operations:** In an attempt to reproduce a realistic utilization of the hash maps, the following operations were performed in a fixed random order accross experiments with the following probabilities:
  - 50% of operations are insertions
  - 40% of operations are searches
  - 10% of operations are deletions
- **Hashing function:** The following basic hashing function was used: `h(k) = k mod m` where k is the key and m is the size of the hash table.
- **Timing:** Timing is performed at the nanosecond level using the `clock_gettime` function with the `CLOCK_MONOTONIC` clock, which is not affected by changes in the system time and provides high-resolution timing. Only the operations are timed to avoid timing overhead from the parts of the code running the experiment, a median is calculated from the 10 runs to get a accurate representation of the time to exclude outliers.
- **Hardware:** The experiments were run on a machine with the following specifications:
  - CPU: Intel Core i7-1360P
  - RAM: 32GB LPDDR5 (Dual Channel, 6000MHz)
  - OS: Windows 11

  In an attempt to reduce as much fluctuation in system performance, the device was put into airplane mode, with all other applications closed and the power settings were set to "Best Performance" to ensure that the CPU is running at its maximum performance level.

Results

The following visualizations show the results of the experiments.

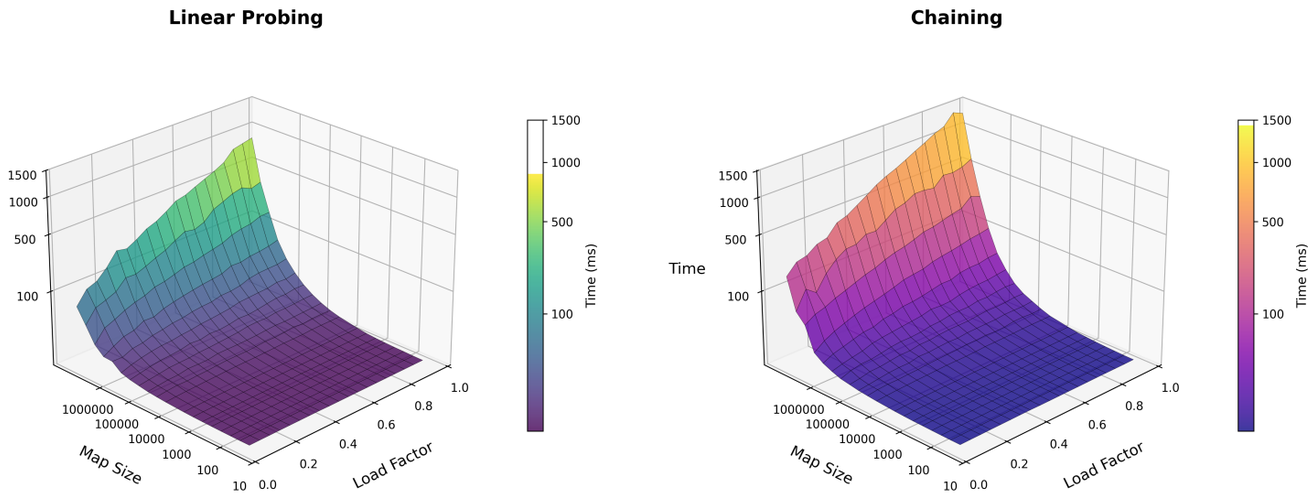
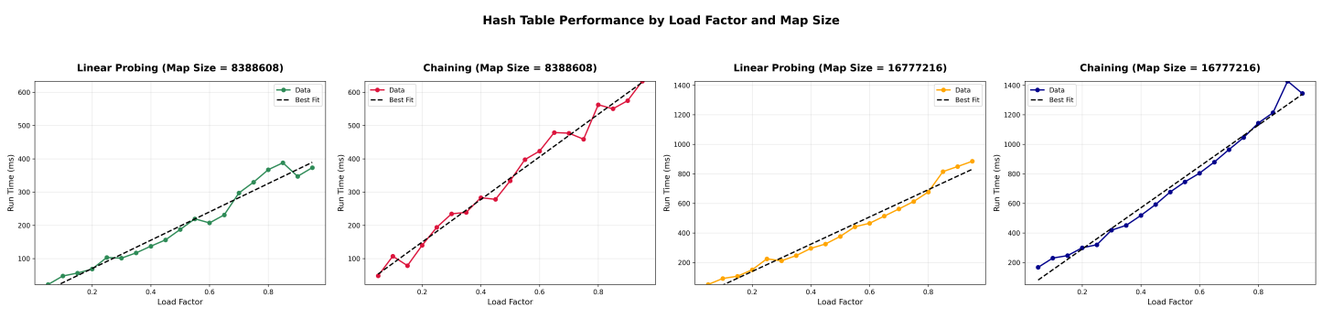
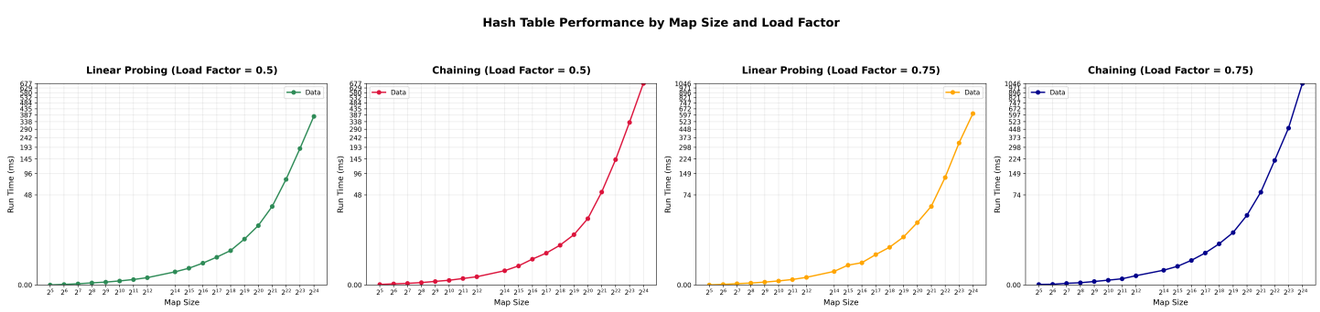


Figure 1: 3D Surface plot comparison of separate chaining and open addressing.

To begin we can look at this 3d surface plot, where the `x` and `y` axes represent the load factor and map size respectively, and the `z` axis represents the time taken for the operations in ms. While this visualization does hide some of the more specific details which we will see later. This gives us a decent overall picture of the performance of the two techniques. We can see in general that linear probing performs better with accross all load factors and map sizes, but the difference is more significant at higher load factors and smaller map sizes. Now we can look at some more specific visualizations to get a better picture of the performance of the two techniques.



Given the following figures, where the `x` axis represents the load factor and the `y` axis represents the time taken for the operations in ms, we can again see that linear probing consistently performs better than seperate chaining, particulary at low to moderate values of α. We can see a slight degredation of performance for linear probing as load factor increases, but not the exponential explotion that we would expect to see. This is most likely due to the combination of different operations being performed and not a sole focus on insertions. Both linear probing and seperate chaining have a relatively linear correlation between load factor and time, but seperate chaining has a much steeper slope, which indicates that the performance of seperate chaining degrades much more significantly as the load factor increases.



Finally we can look at the following figures, where the `x` axis represents the map size and the `y` axis represents the time taken for the operations in ms. We can see that as the map size increases, both linear probing and seperate chaining explode in run time, this is partially due to the exponential nature of the increase in map sizes, and a fractional exponential scaling has been applied to the graphs to compensate for that. We can see that linear probing consistently performs better than seperate chaining, particularly at higher map sizes. The performance of seperate chaining degrades much more significantly as the map size increases.

Conclusion

The goal of this analysis was to compare the performance of seperate chaining and linear probing in a real worl application. The results of the experiments show that linear probing consistently outprerforms seperate chaining across almost all load factors. The most likely reasoning for this result is due to the memory layout of the two techniques, linear probing stores all of the elements in a continuous sequence in memory, which allows for better performance when using that hash tables, where as in seperate chaining each element in the hash table is a pointer to a linked list, which could be scattered anywhere in memory, which can lead to worse performance. The experiment unfortunately does have some holes that were not considered, for example resizing of maps was not considered. Additionally the use of adversarial inputs was not considered which may have demonstrated more significant performance differences. Additionally the use of a singular device for testing may have introduced bias, as well as the use of the windows operating system. Additionally the different operations could have been tested separately to get a better understanding but since we were representing the real world, the mixed operation analysis seemed more fitting. Although it is almost imposible to deterministically say that one algorithm is better then another, due to the wide variety of different uses and parameters. Based on what was experiment it is fair to conclude that on average, linear probing has better overall performance then separate chaining.

References

[1] GeeksforGeeks. (2025a, July 23). *How to find median of numbers in an array in C?* Available at: `https://www.geeksforgeeks.org/c/find-median-of-numbers-in-an-array-in-c/`

[2] GeeksforGeeks. (2025b, July 23). *Program to implement hash table using open addressing.* Available at: `https://www.geeksforgeeks.org/dsa/program-to-implement-hash-table-using-open-addressing/`

[3] GeeksforGeeks. (2026, January 31). *Implementation of hash table in C/C++ using separate chaining.* Available at: `https://www.geeksforgeeks.org/dsa/implementation-of-hash-table-in-c-using-separate-chaining/`

[4] Z. Alessandro. (2020, July 15). *Monotonic clock in C.* Stack Overflow. Available at: `https://stackoverflow.com/questions/62910438/monotonic-clock-in-c`

[5] GeeksforGeeks. (2025c, July 23). *Matplotlib Tutorial.* Available at: `https://www.geeksforgeeks.org/python/matplotlib-tutorial/`

Reflection

a)

Once I got started on this assignment I actually found it alot more enjoyable then I thought I would, it was fun to refresh on my knowledge of C and Python for plotting, but more importantly it was enjoyable to take the time to think about all the factors potentially affecting a experimental analysis of an algorithm. I did find it frustrating at times since it is very difficult to perfectly time an algorithm unless you are measuring individiual instructions in machine code, which bothered me to know my results weren't entirely accurate even if they still gave a good idea of what was going on. I think that this was the first assignment I've done in a DSA course that actually provides some real life experience, which again made the assignment fun to do. In the end it was very rewarding to be able to put together this report after having put in the effort to collect all the data.

b)

1. **Methodology and Thoroughness:** I think that the methodology itself was very good in terms of experimental analysis, but to receive 4 marks it was specified that different approaches needed to be considered, and I only considered the experimental component. I think it could have been beneficial to consider a theoretical analysis alongside the experimental analysis, as the differences between the 2 techniques might have helped draw some more conclusions, regardless, $\boxed{3/4}$

2. **Conclusions:** I believe my conclusion satisified all requirements, the conclusion was clearly connected to the results found through experimental evaluation, and the issues of using only experimental analysis were described aswell. $\boxed{4/4}$

3. **Scientific Context:** I think some reference to the broader context was made but it was not heavily elaborated on and was rather minimal $\boxed{1.5/4}$

4. **Quality and Clarity of Presentation:** The document was already written in the proffesional typesetting language LaTeX, which is a standard for this type of document and I believe that the tone and quality of writing is high. Additionally the visualizations are all cleare and well labeled, which were all generated using Python's Matplotlib library. $\boxed{4/4}$